AI usage log

D6 for the Nirnay demo. How Claude was used, what worked, what was corrected, and the time saved.

Tools

Claude (Claude app with an agent workspace built on Claude Code: web search, code execution, file tools) for research, data generation, coding, interface design, testing with screenshots, writing and deployment. **Gemini on Vertex AI** runs inside the product to read documents and draft notes.

How the work ran

Stage	Key prompt or instruction	What Claude produced
Brief analysis	"Go through the document thoroughly… how do you see it as an analyst?"	Deliverables map, scoring weights (60 of 100 points on research, story and AI use), product recommendation (home loan) and differentiators
Research	Verify RBI rules, CERSAI, KFS, turnaround; use current repo rate	Searches and fetches of RBI and secondary sources; clause pack with verified links; research note
Data	Realistic Indian book: 200 to 1,000 applicants, accounts, loans, repayments	Seeded generator: 600 applications, bureau, CERSAI, KYC, AA, EPFO, GST; four scripted demo files with PDF documents
Lakehouse, agent, app	One well-designed agent; explainable; human in the loop	Raw, curated, gold SQL tested locally then run in BigQuery; seven-step agent, rules with clause ids, Gemini document reading and note drafting, Policy Watch
Interface	"Simple, professional, not AI-looking"	Ledger-style UI; reviewed through screenshots after each pass
Deploy	"Give me the code to run it in my Google Cloud project"	One-command deploy script, run from my own terminal: BigQuery in Mumbai, service account, Gemini model check, Cloud Run
Hand-over and QA	"Check the status, what is pending, and help me finish"	A second Claude session audited the first one's output, re-ran all four demo files, found the open items and wrote the README

What I corrected (with Claude)

- **Wrong citation and guessed links**: the first draft credited LTV caps to the RBI 2020 circular, which sets risk weights only; and early RBI links were guessed IDs. Each was checked against the source and fixed.
- **Unrealistic data**: the first book put 72 of 146 in-flight files outside policy. Rebalanced to a realistic mix and seeded borderline files so the repo-hike story is real, not scripted.
- **Silent bugs**: colliding property IDs gave 606 rows for 600 applications (caught by a row-count check); a field-name clash made GST look missing (caught in a screenshot review).
- **Platform limits**: the build workspace could not reach Google Cloud, and it was unclear which Gemini models serve India. Solved with plain REST clients, a deploy-time model probe and a fallback chain, and by deploying from my own terminal.
- **First deploy failed**: I ran the commands with placeholder IDs still in them. Claude read the error log, gave corrected commands with my real organisation and billing IDs, and the second run went live.
- **Wrong inputs caught**: five JSON files I thought were project outputs turned out to be unrelated health-claim extracts. Claude flagged them before they went anywhere near the submission.

What worked, and what needed steering

Asking for evidence (circular text, row counts, screenshots) instead of accepting first answers worked best. First drafts of copy were wordier than a banker would read, and story-page numbers had to be re-synced with live query results.

Time saved (estimate)

Work	Without AI	With Claude
Research and note	1.5 days	2 hours, plus my review
Synthetic data and documents	1.5 days	1 hour
Backend, agent, SQL	2 days	2 hours
Interface (11 pages)	2 days	2 hours
Total	About 7 working days	About 1 day elapsed

My reflection

Claude proposed most of the build; my job was direction and judgement. I chose the home loan product and the India focus, and pushed for a plain banking interface rather than an AI showcase. When the reference demo turned out to be a knowledge-graph explorer, I asked whether to copy it; we decided not to, because its job is policy search while ours is a loan decision. The biggest lesson was to verify rather than trust: Claude was fast, but the wrong citation, the duplicate rows and the unrelated JSON files were all found by checking. Next time I would talk to bankers in the first two days, not the last, and set up the cloud project before the build.